Reproducing and Improving QUANT: A Minimalist Interval Method for Time Series Classification

Dániel Sajben
University College Dublin
Dublin, Ireland
Student ID: 26107621
daniel.sajben@ucdconnect.ie

Tymur Sukhariev
University College Dublin
Dublin, Ireland
Student ID: 26100566
tymur.sukhariev@ucdconnect.ie

***Abstract*—QUANT is a time series classifier that describes each series by the quantiles of a fixed set of windows and passes them to an off-the-shelf tree ensemble. Its authors report state-of-the-art accuracy for interval methods on 142 UCR datasets in under 15 minutes on one CPU core. We re-ran the published code on the published data and got 85.51% mean accuracy against the paper's 85.68%; the gap comes from four datasets that were corrected after publication. Controlled tests then showed where QUANT is weak: its fixed windows make it sensitive to patterns that appear at different times, and it falls behind on datasets with many classes and few examples. We tested ten ideas to fix this. The best one lets a second, position-free method (Hydra) vote only when QUANT is unsure. It reaches 87.16% (+1.8 points, $p < 0.001$) at 2.1× QUANT's compute time: level with MultiRocket+Hydra at less than half its reported cost, and 0.4 points below HIVE-COTE 2, which needs weeks. Three new views of the series gave small but significant gains, and four ideas did not help. All the ideas we stress-tested made QUANT more robust to moved patterns, but robustness and accuracy turned out to be different things. Code, commands and results: https://github.com/SajbenDani/AI-For-Timeseries-Project.**

***Index Terms*—time series classification, interval methods, quantiles, reproducibility, UCR archive**

I. Introduction

The problem. In time series classification (TSC), each example is a sequence of measurements over time, such as an ECG trace or an accelerometer recording, and the goal is to predict its label. The most accurate TSC methods, such as HIVE-COTE 2 (HC2) [6], take days or weeks of compute on the standard UCR benchmark [3]. This makes them hard to use in practice.

A. Motivation

In a world where the trend is to build bigger and bigger data centres to gain one or two percentage points on a benchmark, QUANT [1], an efficiency focused method, really stood out to us. We see the same habit in how people use language models. Many pick the largest model available (for example Claude Fable over Opus or Sonnet) even for simple tasks, because it is supposed to be the best and they trust it most. The strain on the environment and the size of the bill rarely come up, at least as long as someone else is paying. We think efficiency is key for the future, and we expect the field to keep moving in that direction. Methods that use only the compute a task actually needs, from mixture-of-experts models to small, specialised methods like QUANT, are a good step that way.

Unsurprisingly, QUANT's main strength is efficiency. Instead of adding more parts, the authors stepped back and asked what an interval method actually needs. Their answer was very little: one type of feature (quantiles), a fixed set of windows and an off-the-shelf classifier. What sold the paper to us was the runtime comparison. QUANT needs under 15 minutes on a single CPU core for all 142 datasets. HC2 [6], the most accurate method in the comparison, needs several weeks and scores only about 2 percentage points higher (0.879 vs 0.857). That is exactly the trade-off we see with large language models. We wanted to find out whether we could shrink this gap further, or make QUANT even cheaper without losing accuracy.

The paper also compares QUANT with several other methods, such as MultiRocket+Hydra, rSTSF [7], RDST and HC2. All of them are slower, from about an hour for MultiRocket+Hydra and rSTSF to several weeks for HC2, and the most accurate ones beat QUANT by up to about 2 points. We thought we could take ideas from these methods and build them into QUANT, raising its accuracy while keeping it fast. We also liked that every method is tested on exactly the same 142 datasets and 30 resamples. The comparisons felt transparent, so we believed we had a real chance to improve QUANT in a statistically significant way. That could even help the time series community, which mattered more to us than tweaking random seeds for an extra 0.01% just to finish an assignment.

In short, this is why we chose this paper. We hope you are as curious about our results as we were when we wrote this section, before we started the research.

B. How QUANT Works

QUANT [1] is a classification model: it learns from labelled example series and then predicts the class of new ones. It works in four steps.

1) **Four views.** It looks at every series in four ways: the raw values; the first difference, i.e. how much the series changes from one step to the next (lightly smoothed); the second difference; and the magnitudes of its Fourier transform, i.e. which frequencies it contains.
2) **Fixed windows.** Each view is cut into 1, 2, 4, 8, 16 and 32 equal windows, plus the same windows shifted by half a window.
3) **Quantiles.** In each window QUANT records about one evenly spaced quantile per four values (the median is the 50% quantile, the maximum the 100% one). Every second quantile is taken relative to the window mean, so the features describe both the level and the shape of the values.
4) **Classifier.** A forest of 200 extremely randomised trees (ExtraTrees [5]) learns which of these numbers separate the classes.

For a heartbeat recording, one feature might be "the 90% quantile of the raw values in the second quarter", which is high when a tall beat falls there. Long windows describe *which* values occur, short ones *where*. The transform itself learns nothing; all the learning happens in the forest, which also takes most of the time. A typical dataset gets about 3,000 features. QUANT was taken up quickly: the bake-off redux evaluated it [3], it is part of the aeon toolkit [28], and it is the most accurate method on the large MONSTER datasets [29].

C. The Benchmark

We use the paper's 142 datasets from the "bake off redux" [3]: the 112 equal-length datasets of the 2018 UCR archive [4] and 30 newer ones, all univariate. They cover image outlines turned into series, sensors, motion and gestures, ECG and other medical signals, spectrographs, electricity use, traffic and astronomy. Training sets have 16 to 8,926 series (median 217), series have 15 to 4,767 points (median 331), and there are 2 to 60 classes. Each dataset comes with 30 fixed train/test splits ("resamples"): the original split plus 29 random ones with the same class balance. All methods use the same splits, so results can be compared dataset by dataset. Sixteen new datasets ("Eq") were made equal-length, mostly by padding; our padding tests use the eleven whose originals come with the UCR archive, and Section V-B5 uses three more as unseen data.

D. Where QUANT Is Weak

Our tests (Section IV) found where QUANT loses accuracy:

- **Patterns at different positions.** A pattern that moves lands in a different window and changes the features. With series placed at random positions, QUANT lost 5.6 points of accuracy, Hydra [9] only 2.4.
- **Many classes, few examples.** On 18 datasets with at least 25 classes or at most 5 examples per class, QUANT trails Hydra by 5.6 points, and by up to 27 on the Pig recordings.
- **Padding** costs about 8 points on heavily padded series, though it hurts Hydra just as much.
- **Long series** are slow: QUANT makes about 9 features per time point.

E. Contributions

(1) A scripted reproduction on a laptop CPU, listing what is missing from the paper or has changed since. (2) Controlled tests of where QUANT is weak. (3) Ten improvement ideas tested with the paper's protocol, including a gated QUANT+Hydra ensemble with top-method accuracy at 2.1× QUANT's cost. (4) Negative results and checks on unseen data, reported as they came out.

II. Related Methods

ROCKET [10], MiniRocket [11] and MultiRocket [12] slide thousands of random convolution kernels (small pattern detectors) along each series, summarise every output over the whole series and train a linear (ridge) classifier. Since each summary covers all positions, it does not matter *where* a pattern occurs. Hydra [9], from QUANT's authors, counts which kernel in each group of kernels matches best at every time point, which again ignores position. MultiRocket+Hydra [9] feeds both feature sets into one ridge classifier and came second in the bake-off redux. The winner, HC2 [6], combines four very different classifiers and weights their votes by their estimated accuracy [13]. rSTSF [7] and DrCIF [6] are interval methods like QUANT.

III. Reproducing QUANT

A. Approach

We ran the authors' code [2] unchanged (commit `f3907f7`) through a thin wrapper that also records timings, saves after every resample and fixes the forest's random seed (Python 3.11, PyTorch 2.14 on CPU, scikit-learn 1.9.1). A dataset's accuracy is the mean over its 30 resamples. We compare with the paper's published per-resample results and with an independent QUANT run by the bake-off authors [3], who also provide the competitors' results. A Welch t-test over the resamples tells whether a dataset's gap exceeds resampling noise; across datasets we report the mean, wins/draws/losses (W/D/L) and a Wilcoxon signed-rank test, as the paper does [8].

B. Results

Our reproduction of QUANT. We re-ran the paper's full experiment and got 85.51% vs the paper's 85.68%. The small gap comes from 4 datasets that were corrected after the paper was published (Table I). On the 112 original datasets our mean is 86.71%, the same as the paper's to four decimals; no dataset differs by more than resampling noise (all Welch $p > 0.1$), and our per-resample accuracies track the paper's (median correlation 0.89). Covid3Month and the three FloodModeling sets were corrected after the QUANT preprint appeared, and

TABLE I
MEAN ACCURACY OF QUANT OVER 30 RESAMPLES

Datasets	n	Paper	Bake-off	Ours
Original UCR	112	0.8671	0.8671	0.8671
New, unchanged	26	0.8008	0.8019	0.8017
New, corrected later	4	0.9306	0.8662	0.8673
All	142	0.8568	0.8552	0.8551
All except corrected	138	0.8546	0.8548	0.8548

Paper: published per-resample results. Bake-off: independent QUANT run by [3] on the corrected data.

TABLE II
QUANT VS OTHER METHODS (BAKE-OFF RESULTS): MEAN ACCURACY DIFFERENCE (Δ, POSITIVE = QUANT BETTER), WINS/DRAWS/LOSSES AND WILCOXON p

Method	n	Δ paper	Δ ours	W/D/L ours	p ours
HC2	142	-0.0222	-0.0207	42/7/93	<0.001
MultiRocket+Hydra	142	-0.0165	-0.0163	48/6/88	<0.001
RDST [24]	142	-0.0083	-0.0077	62/4/76	0.133
WEASEL-D [25]	142	-0.0040	-0.0022	69/3/70	0.493
InceptionTime [26]	142	$+0.0001$	-0.0014	64/4/74	0.252
FreshPRINCE [27]	142	$+0.0087$	$+0.0087$	99/3/40	<0.001
Proximity Forest [15]	142	$+0.0267$	$+0.0270$	97/6/39	<0.001
rSTSF	112	$+0.0031$	$+0.0031$	66/8/38	0.022
DrCIF	112	$+0.0035$	$+0.0034$	60/8/44	0.157

we could only download the new versions, on which QUANT scores much lower (Covid3Month: 0.62 vs 0.81). The bake-off authors' own QUANT run on the corrected data matches ours, so the gap comes from the data, not from the method or our code.

Against other methods (Table II), every conclusion of the paper still holds: QUANT is slightly ahead of the best interval methods (significantly only against rSTSF), roughly level with RDST, WEASEL-D and InceptionTime, and clearly behind HC2 and MultiRocket+Hydra. The only change: with the corrected data, QUANT is now marginally behind InceptionTime instead of marginally ahead (not significant either way).

Runtime. One pass over all 142 datasets takes 18.3 minutes on one core of our laptop CPU (AMD Ryzen 5 5600H), against 14.9 minutes on server CPUs in the paper; six datasets with long series cause most of the difference. As the paper says, about 78% of the time goes into training the forest.

C. Lessons Learned

The accuracy claims, the code and the ranking all hold, and reproducing them needs only a laptop CPU. But some details are missing or have changed, and each one cost us time or can quietly change the numbers:

1) **The resamples are not documented.** Two sets of split indices exist, made by the Java and the Python version of the bake-off tool. By correlating per-resample accuracies with the published ones, we found that the paper used the Java set for the 112 original datasets (correlation 0.83–0.96, against about 0 for Python) and the Python set for the 30 new ones.
2) **Dataset names differ** between the code (`...LargeEq`) and the download (`...Large_eq`), so 24 datasets fail without renaming.
3) **The data changed after publication.** Benchmark numbers should come with a recorded data version.
4) **The forest is not seeded**, so results change between runs; single splits of small datasets swing a lot (standard deviation up to 0.09 on Beef).
5) **W/D/L counts depend on rounding** (1–3 datasets move), which the paper does not state.
6) **Two experiments cannot be re-run**: the sensitivity analysis does not list its 50 datasets or folds, and the script uses all cores and records no timings.

Takeaway. The published results hold, but gaps between strong methods are small and noisy, so any improvement has to be tested over many resamples with paired tests.

IV. WHERE QUANT IS WEAK

To decide what to fix, we changed one thing at a time and watched QUANT's accuracy, with Hydra [9] as a reference (Fig. 1).

Patterns at different positions. We placed every series of 30 datasets at a random position inside a window 10%, 25% or 50% longer, filling the gap with the series' first or last value plus a little noise. The patterns stay the same; only their position changes. At 50%, QUANT loses 5.6 points and Hydra 2.4 (Fig. 1a), with the worst cases on outlines of faces and handwritten words (FaceFour -22, WordSynonyms -20). The cause is the fixed windows: a feature such as "the 90th percentile between 25% and 50% of the series" changes when the pattern moves. Real data show the same effect. On the 10 gesture datasets, where people start moving at different times, QUANT trails MultiRocket+Hydra by 5.1 points, against 1.7 elsewhere. Zhang et al. [14] found the same for an older interval method.

Many classes and few examples. Cutting 25 datasets down to 2–20 training examples per class barely changes QUANT's gap to Hydra (0.2 to 1.1 points; Fig. 1b). But on the 18 datasets with many classes *and* few examples per class, QUANT trails Hydra by 5.6 points (Hydra wins 15 of 18, $p = 0.002$), and on the Pig recordings (52 classes, 2 examples each) by 8–27 points. A linear classifier on QUANT's features does even worse, so the forest is not the problem. We think the cause is position again: with two examples per class, the forest cannot learn every place where a heartbeat may sit.

Padding. On the 11 padded datasets, the more heavily padded test series are about 8 points harder for *both* methods, and QUANT's gap to Hydra hardly changes (Fig. 1c). Padding hurts, but it is not what sets QUANT apart.

Other findings. (i) 25 datasets contain test series that are exact copies of training series; removing them costs QUANT up to 5.7 points on a dataset (Worms) but only 0.09 on the mean. (ii) Normalising each series first costs 3.1 points on average, because the absolute level often *is* the class. (iii) The forest slightly neglects small classes on imbalanced

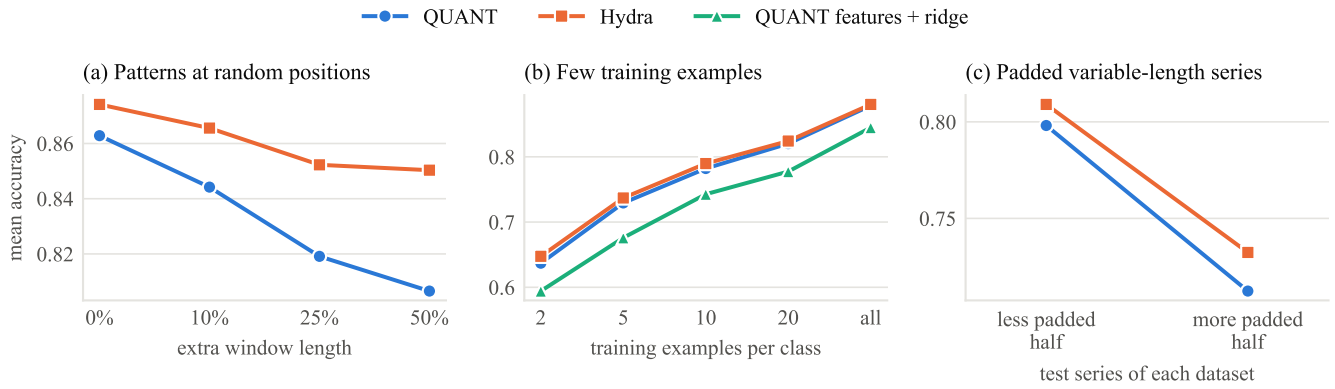


Fig. 1. Where QUANT is weak, with Hydra as a reference. (a) Every series is placed at a random position inside a longer window (30 datasets, 10 resamples). (b) Training sets cut to a few examples per class (25 datasets); "QUANT features + ridge" replaces the forest by a linear classifier. (c) The 11 padded variable-length datasets: accuracy on the less and the more padded half of each dataset's test series.

data. (iv) The 10 datasets with the longest series take 37% of the compute time.

V. Improving QUANT

A. How We Tested

Data and splits. The same 142 datasets and official resamples, but only the first 10 resamples, to fit the runs on a laptop; plain QUANT scores 85.36% on these (85.51% on all 30). The forest's seed is the resample number, so our code reproduces Section III's QUANT accuracies exactly. We never change QUANT: each idea *adds* features, training copies or a second model.

Statistics. Per dataset we average the 10 resamples; across datasets we report W/D/L (four decimals) and a Wilcoxon signed-rank test, with $p < 0.05$ as a real effect. Every gain in Table III's "Worked" group stays significant after a Holm correction for its 14 comparisons; the test-time-averaging loss does not. Inside the training data, idea 3 uses stratified cross-validation (up to 5 folds) and idea 18 the leave-one-out predictions its ridge classifier computes anyway.

Stress tests. On 30 datasets $\times$ 5 resamples: the random-position test of Section IV, and a speed test in which every series is played faster at the start and slower at the end, or the reverse (time $t \mapsto t^g$, $|\log g| \leq 0.6$).

Cost. One-core compute time (same CPU) for features, training and prediction on one split of every dataset, relative to plain QUANT in the same run (about 16 minutes).

Unseen data. The two choices we made after seeing results were tested on data they had not seen (Sections V-B2, V-B5).

B. What Worked

Table III lists every idea and Fig. 2a shows accuracy against cost.

1) A second opinion from Hydra (idea 3): Inspiration: QUANT and Hydra fail on different data, and HC2 shows that averaging different methods pays off. *What:* we train both and average their class probabilities. Hydra's ridge scores become probabilities through a softmax whose "temperature" is fitted on held-out predictions, and each model is weighted by its cross-validated accuracy (HC2's CAWPE rule [13]). *Result:* 87.39%, +2.03 points (99 datasets better, 33 worse, $p < 0.001$), and +6.4 on the 18 many-class datasets (Pig +26 to +27). On the same splits it is statistically level with MultiRocket+Hydra ($+0.33$, $p = 0.37$) and HC2 (-0.13, $p = 0.63$). *Limitation:* 5.8$\times$ QUANT's time (91 instead of 16 minutes), which goes against our own efficiency goal: far cheaper than HC2, but no cheaper than MultiRocket+Hydra. Equal weights give 87.25% at 4.1$\times$. (Correction: our first equal-weights run still cross-validated Hydra's calibration, at

TABLE III
ALL TEN IDEAS: MEAN ACCURACY (%) ON 142 DATASETS, FIRST 10 RESAMPLES. Δ, W/D/L AND WILCOXON p ARE AGAINST QUANT; TIME IS ONE-CORE COMPUTE RELATIVE TO QUANT

Idea	Acc.	Δ	W/D/L	p	Time
QUANT (reference)	85.36				1$\times$
Worked					
3: + Hydra, weighted	87.39	+2.03	99/10/33	<.001	5.8$\times$
3: + Hydra, equal weights	87.25	+1.89	99/7/36	<.001	4.1$\times$
18: + Hydra, two-level gate	87.16	+1.80	83/39/20	<.001	2.1$\times$
18: + small Hydra, gate	86.63	+1.27	84/34/24	<.001	1.3$\times$
1: + pooled quantiles	85.75	+0.39	84/19/39	<.001	1.4$\times$
2: + convolution quantiles	85.76	+0.40	78/7/57	.004	4.0$\times$
14: + speed quantiles	85.50	+0.14	81/9/52	.006	1.3$\times$
10: padded + native*	77.53	+1.97	10/0/1	.005	
same, 3 fresh datasets§	92.52	+0.21	2/0/1		
Did not help					
5: + augmentation	84.64	-0.72	46/9/87	<.001	5.8$\times^{\dagger}$
5: + augm. + test-time avg.	85.01	-0.35	59/10/73	.03	5.9$\times^{\dagger}$
10: native windows + length*	75.66	+0.11	5/0/6	.9	
15: + cycle view	85.36	0.00	61/16/65	.53	1.2$\times$
16: + bispectrum view	85.47	+0.11	65/20/57	.24	1.2$\times$
17: rank-uniform splits	85.27	-0.09	56/10/76	.053	1.2$\times$
MultiRocket+Hydra	87.06	+1.70			$\approx$5$\times^{\ddagger}$
HC2	87.53	+2.17			weeks‡

*The 11 variable-length datasets, 30 resamples (QUANT: 75.55%). §Three variable-length datasets not used before, 30 resamples (QUANT: 92.31%); too few for a test. †Measured on all cores. ‡From [1]; other hardware.

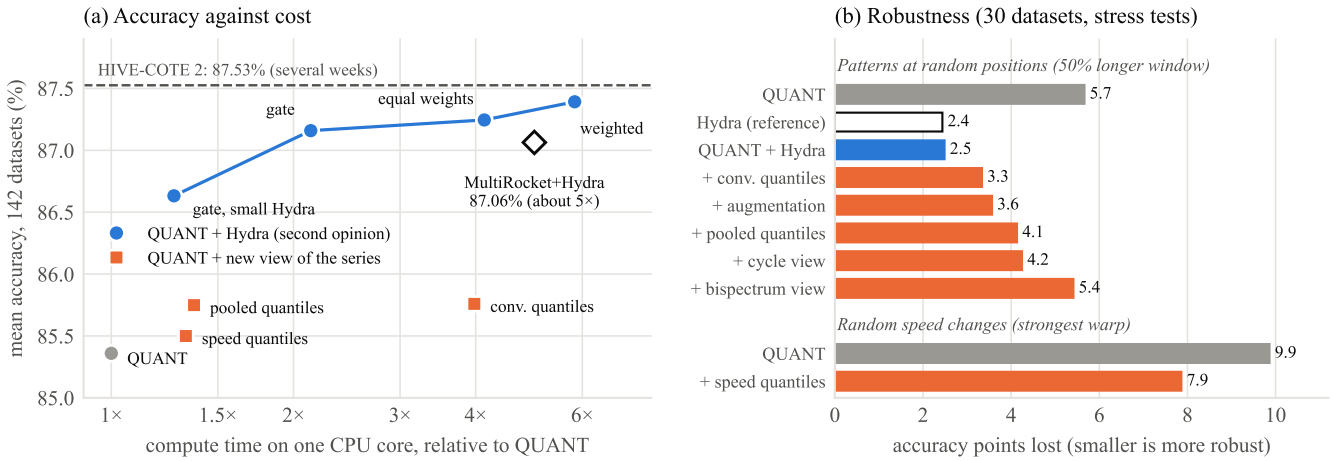


Fig. 2. (a) Mean accuracy against one-core compute time (both relative to plain QUANT in the same timing run). The blue line is QUANT + Hydra with more or less of Hydra used; orange squares add a new view of the series to QUANT. MultiRocket+Hydra's time is from the QUANT paper (other hardware). (b) Accuracy points lost in the two stress tests: patterns at random positions (window 50% longer) and series played at random speeds.

5.0×; free leave-one-out predictions give the same accuracy.)

2) Asking Hydra only when QUANT is unsure (idea 18): Inspiration: the ensemble's gain is concentrated: looking back, 20 datasets gave 66% of it for 6% of Hydra's extra time. *What:* QUANT predicts first; only when the gap between its two most likely classes is below a threshold do we ask Hydra and average. Hydra must process all training series whenever it is used at all, so we also skip it for a whole dataset when too few of its test series are unsure (the *two-level gate*). Settings picked on one half of the datasets are applied to the other half, so no dataset is scored with settings tuned on its own results. *Result:* 87.16%, +1.80 points (83/39/20, $p < 0.001$) at 2.1× QUANT's time: 95% of the ensemble's gain at half its cost. Hydra ran on 66% of the datasets and was asked about 26% of the test series on average. Asking only about unsure series is even slightly better than always asking (87.28% at a gap of 0.5): when QUANT is sure, Hydra can only spoil the answer. As we designed the two-level gate during the run, we also chose its setting on the first 60 datasets alone and tested it on the other 82: it kept 97% of the gain at 2.0× (+2.19 points, $p < 0.001$). With a smaller Hydra (16 instead of 64 kernel groups) it gives +1.27 points for 1.3×, the best accuracy per extra minute we found. *Conclusion:* level with MultiRocket+Hydra (+0.10, $p = 0.43$) at less than half its reported cost (measured on other hardware, so approximate), and 0.37 points behind HC2 ($p = 0.045$). Not a new state of the art, but most of the way there for about twice QUANT's cost.

3) Position-free features (ideas 1 and 2): Inspiration: ROCKET-style methods summarise each pattern detector over the whole series, which is why position does not matter to them. *What:* (a) *pooled quantiles*: slide windows of every size over the series and keep each quantile's maximum, minimum and mean over all positions ("somewhere there is a stretch like this"); (b) *convolution quantiles*: 288 MiniRocket kernels [11], each output summarised by nine quantiles. Both are added to QUANT's features. *Result:* (a) +0.39 points (84/19/39, $p < 0.001$) at 1.4×; (b) +0.40 points (78/7/57, $p = 0.004$) at 4.0×; +1.0 and +1.5 on the many-class datasets. *Conclusion:* the same small gain, but the convolutions cost three times as much, and on their own they are 1.7 points worse than QUANT.

4) Speed-invariant quantiles (idea 14): Inspiration: Proximity Forest [15], which compares series with time-warping distances, is 2.7 points worse than QUANT on average but beats it by more than a point on 23 datasets, mostly gestures and handwriting; QUANT has no notion of speed. *What:* extra window quantiles in which every value counts by how much the series moves there, not by how long it stays, so a level counts as often as the series passes through it [16]. A gesture made twice as slowly gives the same weighted quantiles. *Result:* +0.14 points (81/9/52, $p = 0.006$) at 1.3×, and +0.51 on the 23 datasets where the elastic method wins (18 better, 4 worse, $p = 0.002$). In the speed test it loses 7.9 points instead of 9.9 (Fig. 2b). *Conclusion:* a small gain on the benchmark and a clear gain in robustness to speed changes.

5) Variable-length series: keep the padding, add native windows (idea 10): Inspiration: a padded dataset can contain a series of 11 real points padded to 500, so most of QUANT's windows see only padding. *What:* every window covers the same *fraction* of each series' own length (its first half, its second quarter, ...), which is exactly QUANT on equal-length data. The forest gets these features next to the padded ones. *Result:* 77.53% against 75.55% on the 11 datasets × 30 resamples (+1.97 points, better on 10 of 11, $p = 0.005$). We designed this version after seeing that replacing the padding did not help (Section V-D), so we ran it unchanged on three variable-length datasets we had never used, the Asphalt road-surface sets (30 resamples). There the gain almost disappears: +0.21 points, clear only on AsphaltObstacles (+0.63, better

on 22 of 30 resamples, $p = 0.001$; +0.07 and -0.06 on the others). Native windows alone lost on all three (-1.7). The Asphalt series are padded at least as heavily as the gestures, so the large gains seem specific to gestures, where the same movement is made at different speeds. *Limitation:* our code for the native windows is slow (117 s against 8 s for QUANT's features on the 11 datasets). *Conclusion:* worth adding for gesture-like data, where it gave +2 points; on other variable-length data it costs little accuracy but adds little.

C. Robustness, and What It Means in Practice

Fig. 2b shows the points lost when the same patterns appear at random positions, or when the series are played at random speeds. Every idea we stress-tested lost less than QUANT (the bispectrum view only slightly). The Hydra ensemble loses 2.5 points at 50% offsets, as little as Hydra (2.4) and less than half of QUANT's 5.7, and the speed-invariant quantiles lose 2 points less than QUANT under speed changes. In practice, a robust classifier keeps working when the data are less tidy than the benchmark, for example when a recorder is switched on at a random moment or a movement is made faster than in training. Robustness did not always buy benchmark accuracy, though. The cycle view and augmentation (next section) are more robust but not more accurate, because in most UCR datasets the patterns do not move much.

D. Ideas That Did Not Work

Shift and stretch augmentation (idea 5). *Why:* if moved patterns are the problem, show the forest moved patterns [17]. *What:* four shifted or stretched ($\pm 10\%$) copies of every training series, and averaging over the same copies of each test series. *Result:* -0.72 points ($p < 0.001$), or -0.35 with test-time averaging, at about $6\times$ the cost. It helped where QUANT is weakest (PigAirwayPressure +33) but hurt where position *is* the meaning (Beef -17: a shifted spectrum is a different spectrum). *Lesson:* forcing QUANT to ignore position hurts on average.

Removing the padding (idea 10, first version). *Why:* padding wastes windows. *Result:* with the series length as an extra feature +0.11 points ($p = 0.9$), without it -0.20, with big swings both ways (GesturePebble +3.5, GestureMidAir -6.7). *Lesson:* the padding tells the forest how long, and so how fast, a gesture was.

Cycle view (idea 15). *Why:* QUANT trails HC2 most on series that repeat (-3.9 against -1.5 points elsewhere, $p < 0.001$), and astronomers fold periodic light curves into one cycle [18]. *What:* each series folded into its median cycle, rotated to a standard start. *Result:* ± 0.00 (61/16/65): big gains on a few datasets (ToeSegmentation2 +5.2), losses on outlines (Herring -2.8). It is the most robust new view under random offsets (-4.2 against -5.7), thanks to the standard start rather than the folding.

Shape-aware Fourier view (idea 16). *Why:* QUANT's only position-free view, the Fourier magnitudes, cannot tell a wave from its mirror image. *What:* the bispectrum slice $X(f)^2\,\overline{X(2f)}$, which ignores position but keeps shape [19]. *Result:* +0.11 ($p = 0.24$): +1.0 on gestures, but nothing on the many-class datasets it was meant for.

Rank-uniform splits (idea 17). *Why:* ExtraTrees draws split points uniformly between a feature's extremes, so for a third of QUANT's skewed features most random splits cut off only a few series. *What:* rank every feature before training, a known trick for ordinary trees [20]. *Result:* -0.09 ($p = 0.053$). *Lesson:* picking the best of about 300 candidate splits already makes up for lopsided ones.

E. What We Learned

Four observations tie the results together. First, QUANT's habit of looking at fixed positions is not simply a flaw. On spectra and many sensors, where a value occurs *is* the class, so forcing QUANT to ignore position hurt, while adding a position-free view next to its own helped. Second, a second opinion beat every change to QUANT itself, and it is only needed where QUANT is unsure. Third, fancier features must earn their cost: convolutions gave the same gain as pooled quantiles for three times the price. Fourth, gains appear where the targeted weakness is (many-class data, gestures, datasets where elastic methods win) and get diluted by the many datasets where QUANT is already fine, as our weakness tests predicted.

VI. Ideas We Could Not Try

A tabular foundation model on QUANT's features. *Why it might work:* QUANT's worst case is many classes with few examples, which is what TabPFN [21] and TabICL [22] are built for: pre-trained on millions of synthetic tables, they classify a new dataset in one forward pass. RocketPFN [23] matched HC2 by putting such a model on ROCKET features; nobody has tried QUANT's. *What:* cut QUANT's features down to the model's input limit, use the model instead of the forest, and average the two. *Why not:* at our dataset sizes these models need a GPU, as their memory grows with the number of training series.

Testing at scale. *Why it might work:* QUANT is the most accurate method on the large MONSTER datasets [29], and the two-level gate should save even more there, since Hydra's cost grows with the data. *Why not:* MONSTER needs far more disk space and compute than our laptop's 20 GB budget.

VII. Conclusion

QUANT's published results hold: 85.51% against the paper's 85.68%, with the gap coming from four corrected datasets. Its main weakness is that it looks at fixed positions, which hurts when patterns move and when there are many classes with few examples. Our best fix keeps QUANT and asks Hydra only when QUANT is unsure: 87.16% (+1.8 points) at $2.1\times$ QUANT's time, level with MultiRocket+Hydra at less than half its reported cost and 0.4 points below HC2, which needs weeks. Position-free and speed-invariant views add a little at low cost; several intuitive ideas did not help, and we report them too.

Work Plan and Contributions

Work Plan

Before starting, we met in person and planned the project together. The plan had three steps: reproduce QUANT, find out where it is weak, and only then try to improve it. We wanted each idea to fix a weakness we had measured, not one we had guessed.

The reproduction came first. We ran the authors' code on all 142 datasets and 30 resamples and compared our numbers with the paper's, dataset by dataset. The weakness analysis that followed decided what we worked on next, and it had two parts. We first went through QUANT's published results next to those of the best methods, grouped by type of data, number of classes and training set size. Gesture recordings, datasets with many classes but few examples, and padded datasets stood out. We then ran controlled tests that change one thing at a time, using Hydra as a reference because it does not care where in a series a pattern sits. We moved patterns to random positions, cut training sets down to a few examples per class, compared lightly and heavily padded series, removed test series that also appear in the training set, and normalised the series. The clearest finding: QUANT depends on where things happen in a series.

Our ideas grew out of these results. Every promising one went into an `ideas.md` file, with the weakness it targets and the papers behind it, and we read those papers to judge whether an idea could work. To fit our availability we split the research into two parts, and we met every week to go through new results and decide what to do next. We also set ourselves some rules, mostly so we would not fool ourselves. Every test uses the same splits and seeds as the reproduction and a paired statistical test, failures are reported like successes, and any choice made after seeing results is checked again on data it has not seen.

Work Division

We both agree that the work was divided fairly and equally, 50% each. Both of us did research, coding and evaluation, and we planned the final report together. The research and the conclusions were shared, while the coding and the experiments were divided by time, which suited our timetables. Dániel had more free time early in the semester, so he started with the reproduction, the weakness analysis and the first two rounds of ideas. He left out ideas our hardware could not handle, such as pairing QUANT with a tabular foundation model, which needs a GPU. Tymur had more time later on. He built on Dániel's results, picked up ideas that were still open and extended our tests to multivariate data. He also brought ideas from later lectures into the project and added idea 19 to `ideas.md`. We went through every result together, since we both wanted to learn from the whole project and not only from our own half. Our `ideas.md` lists every idea we discussed, including the ones we never implemented.

Individual Contributions

Dániel Sajben built the reproduction pipeline and the weakness tests and started `ideas.md`. In two rounds he implemented and evaluated ten ideas (1, 2, 3, 5 and 10, then 14–18). Idea 18 worked best: asking Hydra only when QUANT is unsure gave 87.16% at 2.1× QUANT's compute time. Position-free and speed-invariant features added a little, and four ideas did not help. He wrote the first draft of this report and set up the repository so that every number can be checked or rerun.

Tymur Sukhariev took our tests to multivariate data. He set up 19 datasets from the UEA archive [30] and ran QUANT, Hydra and our gated ensemble (idea 18) on them, with up to 5 resamples each. The gate works there too: 75.59% against QUANT's 74.18% (+1.4 points, $p = 0.033$). On the 142 univariate datasets he implemented and evaluated idea 7, autocorrelation and AR features, which gave +0.25 points (not significant, though +14.5 on the repeating PigCVP recordings), and idea 12, counts per window such as how often a series crosses its mean, which made no difference. He also proposed idea 19, views that combine the channels of multivariate data, which we have not tested yet, and made our setup run on macOS.

We both agree on an equal split: Dániel Sajben 50%, Tymur Sukhariev 50%.

REFERENCES

[1] A. Dempster, D. F. Schmidt, and G. I. Webb, "QUANT: A minimalist interval method for time series classification," *Data Mining and Knowledge Discovery*, vol. 38, pp. 2377–2402, 2024.

[2] A. Dempster, "QUANT source code," https://github.com/angus924/quant, 2024.

[3] M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake off redux: A review and experimental evaluation of recent time series classification algorithms," *Data Mining and Knowledge Discovery*, vol. 38, pp. 1958–2031, 2024.

[4] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[5] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[6] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, "HIVE-COTE 2.0: A new meta ensemble for time series classification," *Machine Learning*, vol. 110, pp. 3211–3243, 2021.

[7] N. Cabello, E. Naghizade, J. Qi, and L. Kulik, "Fast, accurate and explainable time series classification through randomization," *Data Mining and Knowledge Discovery*, vol. 38, pp. 748–811, 2024.

[8] A. Ismail-Fawaz, A. Dempster, C. W. Tan, M. Herrmann, L. Miller, D. F. Schmidt, S. Berretti, J. Weber, M. Devanne, G. Forestier, and G. I. Webb, "An approach to multiple comparison benchmark evaluations that is stable under manipulation of the comparate set," arXiv:2305.11921, 2023.

[9] A. Dempster, D. F. Schmidt, and G. I. Webb, "Hydra: Competing convolutional kernels for fast and accurate time series classification," *Data Mining and Knowledge Discovery*, vol. 37, pp. 1779–1805, 2023.

[10] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, pp. 1454–1495, 2020.

[11] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," in *Proc. 27th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2021, pp. 248–257.

[12] C. W. Tan, A. Dempster, C. Bergmeir, and G. I. Webb, "MultiRocket: Multiple pooling operators and transformations for fast and effective time series classification," *Data Mining and Knowledge Discovery*, vol. 36, pp. 1623–1646, 2022.

[13] J. Large, J. Lines, and A. Bagnall, "A probabilistic classifier ensemble weighting scheme based on cross-validated accuracy estimates," *Data Mining and Knowledge Discovery*, vol. 33, pp. 1674–1709, 2019.

[14] Y. Zhang, G. Batista, and S. S. Kanhere, "Revisit time series classification benchmark: The impact of temporal information for classification," arXiv:2503.20264, 2025.

[15] B. Lucas, A. Shifaz, C. Pelletier, L. O'Neill, N. Zaidi, B. Goethals, F. Petitjean, and G. I. Webb, "Proximity Forest: An effective and scalable distance-based classifier for time series," *Data Mining and Knowledge Discovery*, vol. 33, pp. 607–635, 2019.

[16] S. Banach, "Sur les lignes rectifiables et les surfaces dont l'aire est finie," *Fundamenta Mathematicae*, vol. 7, pp. 225–236, 1925.

[17] A. Le Guennec, S. Malinowski, and R. Tavenard, "Data augmentation for time series classification using convolutional neural networks," in *ECML/PKDD Workshop on Advanced Analytics and Learning on Temporal Data*, 2016.

[18] U. Rebbapragada, P. Protopapas, C. E. Brodley, and C. Alcock, "Finding anomalous periodic time series," *Machine Learning*, vol. 74, pp. 281–313, 2009.

[19] P. A. Parker, S. H. Holan, and N. Ravishanker, "Nonlinear time series classification using bispectrum-based deep convolutional neural networks," arXiv:2003.02353, 2020.

[20] T. Galili and I. Meilijson, "Splitting matters: How monotone transformation of predictor variables may improve the predictions of decision tree models," arXiv:1611.04561, 2016.

[21] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, pp. 319–326, 2025.

[22] J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan, "TabICL: A tabular foundation model for in-context learning on large data," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025.

[23] F. M. O'Rourke, A. Trisovic, and D. Bertsimas, "RocketPFN: Accurate time series classification via in-context learning," arXiv:2606.21786, 2026.

[24] A. Guillaume, C. Vrain, and W. Elloumi, "Random dilated shapelet transform: A new approach for time series shapelets," in *Proc. Int. Conf. Pattern Recognition and Artificial Intelligence (ICPRAI)*, LNCS 13363, 2022, pp. 653–664.

[25] P. Schäfer and U. Leser, "WEASEL 2.0: A random dilated dictionary transform for fast, accurate and memory constrained time series classification," *Machine Learning*, vol. 112, pp. 4763–4788, 2023.

[26] H. Ismail Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean, "InceptionTime: Finding AlexNet for time series classification," *Data Mining and Knowledge Discovery*, vol. 34, pp. 1936–1962, 2020.

[27] M. Middlehurst and A. Bagnall, "The FreshPRINCE: A simple transformation based pipeline time series classifier," in *Proc. Int. Conf. Pattern Recognition and Artificial Intelligence (ICPRAI)*, LNCS 13364, 2022, pp. 150–161.

[28] M. Middlehurst, A. Ismail-Fawaz, A. Guillaume, C. Holder, D. Guijo-Rubio, G. Bulatova, L. Tsaprounis, L. Mentel, M. Walter, P. Schäfer, and A. Bagnall, "aeon: A Python toolkit for learning from time series," *Journal of Machine Learning Research*, vol. 25, no. 289, pp. 1–10, 2024.

[29] A. Dempster, N. M. Foumani, C. W. Tan, L. Miller, A. Mishra, M. Salehi, C. Pelletier, D. F. Schmidt, and G. I. Webb, "MONSTER: Monash scalable time series evaluation repository," *Journal of Data-centric Machine Learning Research*, vol. 2, pp. 1–47, 2025.

[30] A. Bagnall, H. A. Dau, J. Lines, M. Flynn, J. Large, A. Bostrom, P. Southam, and E. Keogh, "The UEA multivariate time series classification archive, 2018," arXiv:1811.00075, 2018.